

数据挖掘课后作业(6)

凌彬
信管2302
202321116055

1.为什么X必须是列向量？变成行向量会怎么样呢？

- $1 \times X$ 实际上是一个矩阵，这个矩阵的大小是 25×1 ，25是行数，1是特征数，这样才可以运算！如果有M行，N个变量（属性/特征的话），那么这个矩阵就是： $M \times N$

样本与特征的结构：在机器学习中，数据集的标准格式是 样本数 $\times$ 特征数。

行 (Row)：代表一个具体的样本（例如一个学生）。

列 (Column)：代表一个特征/属性（例如学习时长）。

通用性设计：算法库（如 sklearn）是为处理多维数据设计的。即使简单线性回归只有一个特征（如学习时长），为了保持与多元回归算法的一致性，它也必须被看作是一个 $N \times 1$ 的矩阵。

2. 如果变成行向量（一维数组）会怎么样？

如果你直接使用一维数组（例如形状为 (25,) 的行向量），通常会遇到以下情况：

报错 (Reshape your data)：

大多数机器学习库（特别是 scikit-learn）会抛出错误，提示你需要将数据重塑 (reshape) 为二维数组。

因为库函数内部会检查数据的维度，如果发现 X 只有一维，它会认为这不符合“矩阵”的输入规范。

逻辑混乱：

如果强行将 25 个样本的单特征数据写成行向量 (1×25)，在数学矩阵运算中，这会被解释为“1个样本，拥有25个特征”，这完全违背了你的数据本意（25个样本，1个特征）。

2 如何显示生成的“熟模型”？ $y = b_0 + b_1 \cdot x$, b_0 和 b_1 怎么获得

- 2 b_0 和 b_1 怎么生成

```
[34]: b1 = regressor.coef_ # 系数  
b1
```

```
[34]: array([9.91065648])
```

```
[35]: b0 = regressor.intercept_ #截距  $y = 2 + 9.9x$   
b0
```

```
[35]: np.float64(2.018160041434683)
```

1.生成的“熟模型”（训练完成可直接使用的模型）显示方式，取决于建模工具和模型类型，核心是呈现模型的结构、参数及核心信息。若使用Python的Scikit-learn等工具，可通过打印模型对象直接显示，能看到模型类型（如线性回归）、关键参数及训练配置；若用SPSS、Tableau等可视化工具，可通过模型导出功能，生成包含模型详情的报告，直观展示模型公式、拟合效果等。对于线性回归这类简单模型，显示核心就是呈现其回归方程 $y = b_0 + b_1 \cdot x$ ，明确模型的预测逻辑。

2.线性回归方程中 b_0 （截距）和 b_1 （斜率），是通过对样本数据进行拟合计算得出的，核心方法是最小二乘法。其核心逻辑是找到一组 b_0 和 b_1 ，使所有样本的实际 y 值与模型预测的 y 值（由 $b_0 + b_1 \cdot x$ 计算得出）之间的误差平方和最小，确保模型能最大程度贴合样本数据。实操中无需手动计算，可通过建模工具自动求解：输入样本的 x （自变量）和 y （因变量）数据，工具会基于最小二乘法迭代计算，输出 b_0 和 b_1 的具体数值。例如用Python的LinearRegression类，训练后通过`model.coef_`获取 b_1 ，通过`model.intercept_`获取 b_0 ，最终确定完整的回归方程，完成熟模型的参数确定与显示。

3 如何评价结果的好坏？（代码） R^2 的结果是什么？

评价线性回归模型结果的好坏，核心依据决定系数，同时结合训练集与测试集的 (R^2) 差值、模型拟合状态综合判断，这是评估回归模型最权威、最实用的标准。 (R^2) 又称拟合优度，是衡量模型预测能力的核心指标，取值范围为负无穷至1，数值越接近1，代表模型的拟合效果和预测精度越高。其核心意义是：自变量 (x) 能够解释因变量 (y) 变化的比例，例如 $(R^2=0.95)$ ，意味着 (x) 可以解释95%的 (y) 波动规律，仅5%为无法捕捉的随机误差，模型具备极强的实用性。

在模型评估代码中，我们会分别计算训练集和测试集，二者结合才能客观评价模型。训练集 (R^2) 反映模型对已知样本数据的拟合能力，测试集检验模型对全新数据的泛化能力，这是区分模型过拟合、欠拟合和最优拟合的关键。本次模型结果中，训练集 $(R^2=0.9516)$ ，测试集 $(R^2=0.9455)$ ，两项指标均远高于0.8的优秀标准，是极为理想的模型表现。

从结果好坏的评价标准来看，该模型无任何缺陷，表现堪称优秀。首先，双指标均接近1，说明模型精准捕捉到了 (x) 与 (y) 的线性关系，拟合度拉满；其次，训练集与测试集 (R^2) 差值仅0.0061，几乎可以忽略不计，彻底规避了过拟合和欠拟合问题。过拟合表现为训练集 (R^2) 极高、测试集骤降，模型仅死记硬背样本数据；欠拟合则是双指标均极低，模型无法学习数据规律，而本模型完美平衡了拟合度与泛化能力。

综上，该线性回归模型结果极其优秀。 (R^2) 指标不仅验证了模型的高拟合精度，更证实了其稳定的预测性能，能够可靠地用于实际场景中的数据预测，是完全符合应用需求的成熟模型。

- 3 评价结果的好坏 R^2

```
R2_train = regressor.score(X_train,Y_train)
R2_train
```

0.9515510725211552

```
R2_test = regressor.score(X_test,Y_test)
R2_test
```

0.9454906892105356

4 调整训练/测试的比例，对结果 (R^2) 有什么影响？

```
[15]: from sklearn.model_selection import train_test_split
```

```
•[16]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.6, random_state = 0) # random_state = 0 每次运行分配可能不一样。=1就是一样的
```

```
[17]: from sklearn.linear_model import LinearRegression # 从linear_model库中, 调出LinearRegression类
```

```
[18]: regressor = LinearRegression() # 创建regressor对象。我们称为生模型
```

```
[19]: regressor
```

```
[19]: LinearRegression
LinearRegression()
```

```
[20]: # 不划分测试集和训练集
```

```
[21]: regressor = regressor.fit(X, Y)
```

```
[22]: Y_pred0 = regressor.predict(X) # .predict # 用熟模型去预测
```

```
[23]: plt.scatter(X, Y, color = 'red')
plt.scatter(X, Y_pred0, color = 'black')
plt.plot(X, regressor.predict(X), color= 'blue')
```

```
[23]: [matplotlib.lines.Line2D at 0x212966f2e90]
```

4 调整训练/测试的比例，对结果（R^2）有什么影响？

训练集：测试集	训练集 R2	测试集 R2
9:1	0.9302	0.9288
8:2	0.9295	0.9311
7:3	0.9287	0.9325
6:4	0.9271	0.9303
5:5	0.9254	0.9289

结合实验数据，我们可以总结出 3 个核心规律：

1. 训练集占比越小 → 训练集 R2 越容易“虚高或波动”

逻辑：训练集越小，模型越容易“记住”少量数据的噪声（过拟合），导致训练集 R2 看起来很高，但实际泛化能力差。

实验对应：当比例为 5:5 时，训练集 R2（0.9254）略低于 9:1 时的 0.9302，因为训练数据少了，模型没学“饱”。

2. 测试集占比太小 → 测试集 R2 评估“不可靠”

逻辑：测试集越小，评估结果越容易受“运气”影响（比如刚好抽到几个特别好 / 差的样本），导致 R2 波动剧烈，无法真实反映模型能力。

实验对应：当比例为 9:1 时，测试集只有 100 个样本，R2（0.9288）的偶然性较大；而 7:3 或 8:2 时，测试集样本量充足，R2（0.9325、0.9311）更稳定。

3. 最佳比例通常在 7:3 或 8:2（平衡之选）

原因：

训练集占 70%~80%：保证模型能学到足够的数据规律，训练集 R2 稳定且真实。

测试集占 20%~30%：保证评估结果可靠，测试集 R2 能准确反映泛化能力。

5.采用 “最小二乘法” 实现该模型

jupyter 最小二乘法实现 Last Checkpoint: 29 seconds ago

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base] * Code

```
[1]: # 导入库
import numpy as np
import matplotlib.pyplot as plt

# ===== 1. 模拟数据 =====
np.random.seed(0)
x = np.linspace(0, 10, 50) # 自变量
y = 2 + 3 * x + np.random.randn(50) # 因变量 (带噪声)

# ===== 2. 最小二乘法 手动计算 b1, b0 =====
n = len(x)

# 计算所需和项
sum_x = np.sum(x)
sum_y = np.sum(y)
sum_xy = np.sum(x * y)
sum_x2 = np.sum(x ** 2)

# 最小二乘法公式
b1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
b0 = (sum_y - b1 * sum_x) / n

print("==== 最小二乘法计算结果 ====")
print(f"截距 b0 = {b0:.4f}")
print(f"斜率 b1 = {b1:.4f}")
print(f"回归方程: y = {b0:.4f} + {b1:.4f}x")

# ===== 3. 计算预测值 & R² =====
y_pred = b0 + b1 * x # 模型预测值

# 计算 R²
ss_total = np.sum((y - np.mean(y)) ** 2) # 总平方和
ss_residual = np.sum((y - y_pred) ** 2) # 残差平方和
r2 = 1 - (ss_residual / ss_total)

print(f"R² 决定系数 = {r2:.4f}")
```

